

S3 + CloudFront — Practical System Design Notes

Hinglish • Common-sense explanations • Interview-focused

1. The Core Mental Model

S3 = object storage, CloudFront = CDN/cache, Node.js = API/orchestration, Browser = first cache layer.

```
User
↓
Browser Cache
↓ cache miss
CloudFront Cache
↓ cache miss
S3 (origin/storage)
```

2. S3 Pricing: Four Different Meters

Do not think that paying for 100 GB means unlimited free downloads. S3 billing has separate dimensions.

Meter	What it means	Example
Storage	How much data is kept	100 GB stored
Requests	How many API operations	PUT, GET, DELETE, LIST
Data transfer	How many bytes move	100 GB downloaded
Other features	Extra services/configuration	Replication, lifecycle, etc.

Upload: data coming into S3 generally does not incur a standard internet data-transfer-in charge, but API requests can be charged. **Download:** data leaving AWS can incur applicable data-transfer charges. Request charges and transfer charges are separate.

Example:

100 GB stored
≈ storage charge

100 GB downloaded
≈ additional data-transfer charge

1,000,000 PUT requests
≈ request charge, independent of total object size

3. Why 1 Large File vs Many Small Files Matters

S3 request pricing is based on request count/type, not simply the number of bytes in each object. So one large object can have far fewer request operations than millions of tiny objects.

Conceptual example:

1 × 5 GB video
→ roughly one simple PUT operation (multipart may use several operations)

1,000,000 × 5 KB images
→ 1,000,000 object upload operations

The tiny files can create much higher request cost even when total data is small.

Important: a real 5 GB upload may use multipart upload, which introduces additional multipart API operations. Do not memorize '5 GB = exactly one request'; memorize 'request count is separately metered'.

4. Presigned URL

For large uploads, avoid making the Node.js API carry the entire file. Node.js can authenticate the user and generate a temporary S3 presigned URL; the browser then uploads directly to S3.

Bad for large files:

User → Node.js → S3

↑
500 MB passes through API

Better:

User → Node.js

↓
presigned URL
↓

User ■■■■■■■■■■ → S3
500 MB

Why? The API avoids unnecessary network bandwidth and connection handling. Memory usage also depends on whether the server streams or buffers the upload; the key concern is that the large file does not need to traverse the Node.js API.

5. Multipart Upload

Large files can be split into parts. Parts can upload in parallel, and if one part fails, that part can be retried instead of restarting the entire upload.

5 GB video

[1 GB] [1 GB] [1 GB] [1 GB] [1 GB]
✓ ✓ ✗ ✓ ✓

Retry only failed part.

After CompleteMultipartUpload:

S3 exposes ONE 5 GB object, not five separate videos.

Incomplete multipart uploads can leave uploaded parts behind. Use an S3 Lifecycle Rule to abort incomplete multipart uploads after an appropriate period.

6. Node.js Role in Direct-to-S3 Multipart Upload

Node.js handles:

- Authentication / authorization
- Start multipart upload
- Generate presigned URLs
- Track upload metadata
- Complete or abort the multipart upload

Browser/client handles:

- Actual file transfer
- Uploading parts directly to S3

7. CloudFront: Why Use It?

CloudFront is a CDN/cache layer in front of an origin such as S3. It can serve cached content close to users, reduce repeated origin fetches, absorb traffic spikes, and improve delivery latency for distributed users.

Without CDN:

User 1 ■ → S3
User 2 ■ → S3
User 3 ■ → S3

With CloudFront:

User 1 ■■
User 2 ■■ → CloudFront → S3 on cache miss
User 3 ■■

CloudFront does not make data transfer disappear. Users still receive bytes; CloudFront changes where those bytes are served from and reduces repeated origin fetches.

8. Cache HIT vs MISS

First request:

User → CloudFront → MISS → S3 → CloudFront → User

Later request while cached:

User → CloudFront → HIT → User

Example:

1 GB object + 100 users

If the object stays cached:

S3 → CloudFront ≈ 1 GB

CloudFront → users ≈ 100 GB

If the cache expires, the next request may trigger an origin fetch or revalidation. TTL expiry does not mean CloudFront runs a cron job and immediately fetches the object.

9. Cache Key

A cache key is the information CloudFront uses to decide whether two requests should map to the same cached response. Depending on configuration, path, query strings, headers, and cookies can affect cache behavior.

```
/image.jpg?size=small  
/image.jpg?size=large
```

If the relevant query parameter is part of the cache key, these can become separate cache entries.

Common-sense rule: **Cache Key = 'How does CloudFront decide that these two requests are asking for the same cached content?'**

10. TTL — Time To Live

TTL tells the cache how long a response can be considered fresh under the configured caching policy.

TTL = 10 minutes

```
10:00 → object fetched and cached  
10:01 → HIT  
10:05 → HIT  
10:10 → cached freshness expires  
10:11 → next request may require origin fetch/revalidation
```

TTL is not a cron job. The cache does not need to contact S3 simply because the clock crossed the TTL boundary; the next relevant request drives what happens.

11. Cache Invalidation

Invalidation is an explicit way to remove cached content before its normal TTL would expire.

S3:
product.jpg = NEW IMAGE

CloudFront:
product.jpg = OLD CACHED IMAGE

Invalidate:
/product.jpg

Next request:
CloudFront MISS → S3 → NEW IMAGE

If only one object changed, invalidate the specific path rather than unnecessarily invalidating the whole distribution.

12. Versioned / Immutable Assets

For assets that can be versioned, changing the filename is often cleaner than repeatedly invalidating a long-lived cache entry.

Old:
logo-v1.png

New:
logo-v2.png

or:
app.a82f31.js
app.b91c42.js

New URL = new cache key.

This is why hashed static assets can use long browser/CDN cache lifetimes. The exact cache policy should match your deployment and invalidation strategy.

13. Browser Cache vs CloudFront Cache

Request path:

User
↓
Browser Cache
↓ MISS
CloudFront Cache
↓ MISS
S3

Possible outcomes:

Browser HIT → request may stop at browser
Browser MISS + CloudFront HIT → origin not contacted
Browser MISS + CloudFront MISS → S3/origin involved

Browser cache and CloudFront cache are independent layers. A browser miss does not imply a CloudFront miss.

14. Cache-Control: public vs private

Public-cacheable content: same response can safely be shared, such as many public product images or versioned static assets. **Private content:** user-specific responses should not be reused by a shared cache. For sensitive responses, policies such as private or no-store may be appropriate depending on requirements.

Public:
Cache-Control: public, max-age=86400

Private:
Cache-Control: private

Sensitive/no storage when required:
Cache-Control: private, no-store

Important nuance: 'public/private' describes shared-cache behavior. An endpoint being authenticated does not by itself prove that every response must be uncacheable; the actual response semantics and cache configuration matter.

15. Private Content + Signed URLs

For private content, an application can authenticate the user and issue short-lived authorization for delivery. When CloudFront is the delivery layer, CloudFront signed URLs/cookies can restrict access to private content.

Private video:

```
User → Node.js
      ↓ authenticate / authorize
      ↓ signed CloudFront URL
User → CloudFront → S3
```

Do not confuse an S3 presigned URL with a CloudFront signed URL: they protect different access paths. S3 presigned URLs grant temporary access to S3 operations; CloudFront signed URLs/cookies authorize delivery through CloudFront.

16. S3 Security

A strong default is to keep the S3 bucket private and explicitly control access. Do not make an entire bucket public just because some content is public.

Example logical layout:

```
bucket/
  ■■■ public/
  ■   ■■■ logo.png
  ■   ■■■ products/123.jpg
  ■■■ private/
  ■■■   invoices/
  ■■■   user-videos/
```

Default:

S3 Block Public Access / private bucket

Public delivery:

CloudFront can serve intended public content

Private delivery:

Authenticated access + signed authorization

17. S3 Storage Classes

S3 offers storage classes for different access patterns. Standard is for frequently accessed data; IA classes target less-frequent access; Glacier classes target archival use cases. Cheaper storage can come with retrieval charges, minimum-duration considerations, slower retrieval, or different availability/redundancy characteristics depending on the class.

Class family	Common use
S3 Standard	Frequently accessed objects
S3 Standard-IA	Infrequently accessed objects
S3 One Zone-IA	Infrequent access where single-AZ storage is acceptable
Glacier Instant Retrieval	Archive with fast retrieval needs
Glacier Flexible Retrieval	Archive where retrieval can take longer
Glacier Deep Archive	Very rarely accessed long-term archive

18. Lifecycle Rules

Lifecycle rules automate transitions and cleanup. You do not need a Node.js cron job just to move objects between S3 storage classes.

Example concept:

```

New object
↓
S3 Standard
  ↓ lifecycle transition
S3 Standard-IA
  ↓ lifecycle transition
Glacier class

```

Lifecycle can also:

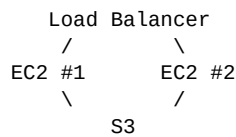
- Abort incomplete multipart uploads
- Expire old object versions
- Delete objects after a defined retention period

Choose transition timing based on the real access pattern and the pricing/retention characteristics of the selected class.

19. S3 vs EBS vs EFS

Storage	Mental model	Typical use
EBS	EC2's attached block disk	OS, application disk, database/block storage
EFS	Shared network filesystem	Multiple EC2 instances need a shared filesystem
S3	Object storage / warehouse	Images, videos, documents, backups, large object collections

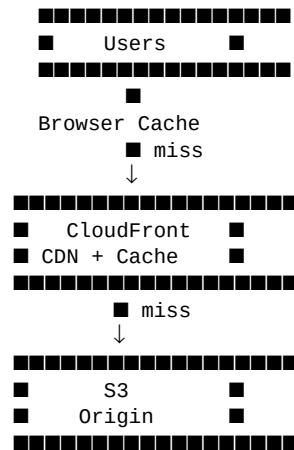
Multiple EC2 servers:



If a file is stored only on EC2 #1's EBS, a request routed to EC2 #2 may not find it.

Use S3 for object storage, or EFS when a shared filesystem is actually required.

20. Complete Production Mental Model



UPLOAD:

User → Node.js (auth / upload setup)
 → presigned URL
 → S3 directly
 → multipart for large files

DOWNLOAD:

User → Browser cache
 → CloudFront cache
 → S3 on miss/revalidation

PRIVATE:

Authenticated user
 → short-lived signed authorization
 → CloudFront
 → private S3 origin

21. Interview Cheat Sheet

Question / Concept	Short answer
S3 storage vs traffic	Storage and data transfer are independent. 100 GB stored can serve many TB of downloads.
PUT vs file size	Request charges depend on request count/type. Large multipart uploads can involve multiple API requests.
Presigned URL	Temporary permission so the client can access S3 directly without routing the large file through Node.js.
Multipart upload	Splits large uploads into parts; supports retrying failed parts and parallelism.
CloudFront	CDN/cache layer that reduces repeated origin fetches and improves global delivery.
Cache HIT	Serve cached response; origin is generally not contacted for that request.
Cache MISS	Fetch/revalidate from origin, then serve/cache according to policy.
TTL	Freshness period for cached content; not a cron job.
Invalidation	Explicitly remove cached paths before normal expiry.
Versioned assets	New filename/hash creates a new cache key and avoids stale old content.
Public vs private	Public content can be shared through caches; private/user-specific data should not be shared incorrectly.
Lifecycle	Automatically transition or expire objects; can clean incomplete multipart uploads.
EBS	Block storage attached to EC2.
EFS	Shared filesystem for multiple compute instances.
S3	Object storage for files, media, backups, and large object collections.

22. Final Mental Model

Remember these four words:

STORAGE → How much data exists?

REQUESTS → How many API operations?

TRANSFER → How many bytes move?

CACHE → Can we avoid going to the origin again?

And:

S3 → Object storage

CloudFront → CDN / cache

Presigned URL → Temporary S3 access

Signed URL/Cookie → Private CloudFront delivery

Lifecycle → Automatic storage/cleanup rules

EBS → EC2 block disk

EFS → Shared filesystem